MSX-BACON

MSX-BASIC

CROSS COMPILER

MSX-BACON USER'S MANUAL

2026.10

目次

項目	ページ
第 1 章 MSX-BACON のコマンドラインでの使い方	3
第 2 章 コンパイル結果の実行方法	10
第 3 章 MSX-BASIC での組み方の解説	13
第 4 章 対応命令リファレンス	20
命令索引	62

第 1 章 MSX-BACON のコマンドラインでの使い方

1.1 MSX-BACON とは

MSX-BACON は、MSX-BASIC のプログラムをコンパイルして機械語に変換するクロスコンパイラである。MSX-BASIC の文法で書かれたプログラムを入力すると、Z80 アセンブリ言語のソースコードを出力する。出力されたアセンブリ言語をクロスアセンブラ ZMA でアセンブルすると、MSX 上で実行できるバイナリファイル（BLOAD 形式）が得られる。

処理の流れは次のとおりである。

段階	入力	処理	出力
1	BASIC プログラム（.bas）	MSX-BACON でコンパイル	アセンブリ言語（.asm）
2	アセンブリ言語（.asm）	ZMA でアセンブル	バイナリ（.bin）
3	バイナリ（.bin）	MSX 上で BACONLDR.BIN と共に読み込む	実行

コンパイルしたプログラムは単独では動作せず、実行には MSX 側に常駐ライブラリ BACONLIB が必要である。ライブラリの設置と実行方法は第 2 章で述べる。対応している命令・関数の一覧と詳細は第 4 章、MSX-BASIC との違いを踏まえたプログラムの書き方は第 3 章を参照されたい。

本書は MSX-BACON v0.9.2 を対象とする。v0.9.2 は、v0.9.1 以降の改善（正しさの修正、生成コードのサイズ最適化、ファイル入力系の完成、式・文の不足分の補完、DEF FN / ON ERROR GOTO / DRAW への対応）をまとめた版である。変更の概要は 1.7 節にまとめている。なお、SQR 関数と中間言語形式の読み込みについては、前版の v0.9 では不具合のため本書の記述どおりに動作しなかったもので、v0.9.1 で修正された挙動を引き続き記している（1.5 節、第 4 章の SQR の項）。

1.2 必要なもの

MSX-BACON の利用には次のものが必要である。

必要なもの	用途	備考
MSX-BACON 本体（msx_bacon）	BASIC からアセンブリ言語への変換	ソースコードからビルドするか、付属のビルド済みバイナリを使う
ZMA	アセンブリ言語からバイナリへの変換	出力は ZMA の文法で書かれているため、他のアセンブラではアセンブルできない
BACONLDR.BIN	MSX 側でライブラリを設置するローダー	リポジトリの msx_basic_compiler/baconloader/ にある
MSX 本体（またはエミュレータ）	コンパイル結果の実行	Disk BASIC が使える環境が必要

コンパイラ本体は Windows / macOS / Linux で動作する。ソースコードは C++ で書かれており、リポジトリの msx_basic_compiler フォルダで make によりビルドできる。

コンパイラとローダー（BACONLDR.BIN）は、必ず同じ世代のものを組み合わせて使うこと。世代が食い違うと、コンパイルしたプログラムが正しく動作しないことがある。特に PLAY 命令を含むプログラムは、PLAY 対応以降のコンパイラで再コンパイルし、新しいローダーと組み合わせる必要がある。

1.3 コンパイルの手順

BASIC プログラム hello.bas をバイナリ hello.bin に変換する手順を示す。

まず、BASIC プログラムを用意する。

```
10 PRINT "HELLO! BACON"
20 GOTO 10
```

MSX-BACON でコンパイルし、アセンブリ言語 hello.asm を得る。

```
msx_bacon hello.bas hello.asm
```

実行すると、次のような表示が出る。

```
MSX-BACON v0.9.2
============================================================
```

```
Copyright (C)2023 t.hara
Compile mode: MSX-BASIC Compatible.
Optimization level: O2 (Code)
Start address: 0x8010
Stack size: 256

Target: hello.bas (ASCII code).
Completed.
```

「Completed.」と表示されればコンパイル成功である。次に、ZMA でアセンブルしてバイナリを得る。

```
zma hello.asm hello.bin
```

得られた hello.bin を BACONLDR.BIN と一緒に MSX へ転送し、第 2 章の手順で実行する。

コンパイルでエラーが見つかった場合は、.asm ファイルは出力されない（1.6 節）。

1.4 オプション

コマンドラインの書式は次のとおりである。

```
msx_bacon [オプション] <入力.bas> <出力.asm>
```

指定できるオプションは次の 5 種類である。オプションは入力ファイル名の前に書く。

オプション	意味	既定値
-compatible	MSX-BASIC 互換モードでコンパイルする	（既定はこちら）
-original	MSX-BACON 拡張モードでコンパイルする	
-O0 / -O1 / -O2 / -O3	最適化レベルを指定する	-O2
-start n	プログラムの開始アドレスを指定する	0x8010
-stack n	呼び出しスタックのサイズを指定する	256

知らないオプションを指定すると「ERROR: Unknown option」と表示して終了する。

1.4.1 コンパイルモード（-compatible / -original）

- -compatible は MSX-BASIC 互換モード、-original は MSX-BACON 拡張モードである。既定は -compatible である。
- 両モードの違いは変数名の扱いだけである。-compatible では MSX-BASIC と同じく変数名は先頭 2 文字までが有効で、ABC と ABD は同じ変数として扱われる。-original では変数名の全体が有効になり、長い変数名をそのまま使える。詳しくは 3.2 節を参照されたい。

1.4.2 最適化レベル（-O0 〜 -O3）

レベル	内容
-O0	最適化を行わない
-O1	演算ノードの事前計算（定数畳み込み）のみを行う
-O2	さらに、生成コードの再構成を行う
-O3	最大の最適化を行う

既定は -O2 である。最適化によってプログラムの意味が変わることはないため、通常は既定のままでよい。生成コードを読みながら調整したい場合や、問題の切り分けをしたい場合に -O0 を使うとよい。

1.4.3 開始アドレス（-start）

プログラムコードの開始アドレスを指定する。既定は 0x8010 である。

- 数値は 10 進数で指定する。「0x8010」のような 16 進表記は解釈されない。
- 32768 未満を指定すると 32768 に、65535 より大きい値を指定すると 65535 に、それぞれ丸められる。

開始アドレスは、コンパイル結果を読み込むメモリ配置を決めるものである。通常は既定のままでよい。他の機械語プログラムと共存させる場合などに変更する（2.6 節も参照）。

1.4.4 スタックサイズ（-stack）

GOSUB や関数呼び出しに使う呼び出しスタックのサイズを指定する。既定は 256 である。

- 0 未満は 0 に、16384 より大きい値は 16384 に丸められ、最終的に偶数に調整される。

- スタックは文字列の一時領域の個数も決める（サイズの約 1/3 個分、最小 8 個）。深い再帰や大きな GOSUB の入れ子を使うプログラムでは大きめに指定する。

1.5 入力ファイルの形式

入力の BASIC プログラムは、テキスト形式（ASCII 形式）と、MSX-BASIC が保存する中間言語形式（トークン形式）の両方に対応している。形式は自動で判別されるため、指定は不要である。ファイルの先頭バイトが FFh であれば中間言語形式、それ以外は ASCII 形式として読み込まれる。

どちらの形式で読み込んだかは、コンパイル時の「Target: ファイル名 （形式）」の表示で分かる。ASCII 形式は「ASCII code」、中間言語形式は「Precompiled code」と表示される。中間言語形式のプログラムは、読み込み時にテキストへ変換されてからコンパイルされるため、等価な ASCII 形式のソースと同じコンパイル結果が得られる。

なお、配布された v0.9 では中間言語形式の読み込みに不具合があり、&B 形式の定数を含むプログラムで処理が止まってしまう、実数定数や数字の 9 を含む数値が化ける、といった問題があった。これらは v0.9.1 で修理されている。

プログラムには行番号が必要である。行番号のないソースはエラーになる。行番号の代わりに（あるいは行番号と併用して）ラベルも使える。書式の詳細は 3.1 節を参照されたい。

1.6 コンパイルエラーと終了コード

コンパイル中にエラーが見つかると、標準エラー出力に次の形式で表示される。

```
ERROR(行番号): エラーメッセージ
```

エラーが 1 件でもあると、最後に「Found N error(s).」と表示され、.asm ファイルは出力されない。主なエラーメッセージは次のとおりである。

メッセージ	意味
Syntax error.	構文エラー。文法に合わない記述、対応していない命令の使用など
Missing operand.	引数や式が足りない
Illegal function call.	引数が不正、または値が範囲外である
Type mismatch.	型が一致しない

メッセージ	意味
Redimensioned array.	確保済みの配列を再度 DIM した
Undifined line number.	飛び先の行番号が存在しない（綴りは原文のまま）
Subscript out of range.	配列の添字が範囲外である（コンパイル時に分かる場合）
Overflow.	数値が範囲を超えた
String too long.	文字列が長すぎる（255 バイトを超える定数文字列など）

このほか、入力ファイルが開けない・読めない・空である場合は、その旨のメッセージが表示される。

終了コードは次のとおりである。

終了コード	意味
0	正常終了
1	コンパイルエラーがあった、またはオプションの指定が不正
2	入力ファイルの読み込み、または出力ファイルの作成に失敗

1.7 バージョン履歴

本書が対象とする版までの主な変更は次のとおりである。

版	主な変更
v0.9.2	正しさの修正（O2 / O3 でアセンブルできない出力の生成、GET DATE / GET TIME の ,A オプション、OPEN の APPEND 動作、BASE の定数表の誤り、IF ～ THEN 行番号の直後の GOTO が消える問題、LPRINT の出力がプリンタへ届かない問題）、生成コードのサイズ最適化（処理の BACONLIB への追い出し、直接ストア化など）、ファイル入力系の完成（INPUT #、EOF、LOC、LOF、DSKF、INPUT$ のファイル形、GET # / PUT # のレコード番号省略、RESTORE の * ラベル形）、式・文の補完（式中の SPC、MAX、INSTR の 3 引数形、SPRITE$ の読み取り、POS、LPOS、PDL、FRND、LEN の数値形、BASE 代入形、ABS(-32768) の型昇格）、DEF FN、ON ERROR GOTO /

版	主な変更
	RESUME、DRAW、LINE INPUT、WAIT、SET DATE / SET TIME への対応
v0.9.1	SQR 関数の登録漏れの修正、中間言語形式（Precompiled code）の読み込みの修理
v0.9	前版

第 2 章 コンパイル結果の実行方法

2.1 出力ファイルの形式

ZMA でアセンブルして得られるバイナリファイルは、MSX の BSAVE（BLOAD）形式である。先頭に次のヘッダが付いている。

オフセット	内容
+0	識別バイト 0FEh
+1 ～ +2	ロード開始アドレス
+3 ～ +4	終了アドレス
+5 ～ +6	実行開始アドレス

ロード開始アドレスと実行開始アドレスは、コンパイル時の開始アドレス（既定は 8010h、1.4.3 節）である。終了アドレスは生成コードの末尾位置である。

2.2 BACONLIB とローダー

コンパイルしたプログラムは、実行時ルーチンの集まりであるライブラリ BACONLIB を呼び出して動作する。プログラム単独では動作しないため、実行の前にライブラリを MSX 上に設置する必要がある。

ライブラリの設置はローダー BACONLDR.BIN が行う。BACONLDR.BIN を実行すると、内蔵している BACONLIB のイメージ（約 9.2KB）を RAM のページ 1（4000h ～ 7FFFh）にコピーして常駐させ、画面に次のメッセージを表示して BASIC に戻る。

```
MSX-BACON Library installed.
Copyright (C)2023 HRA!
```

コンパイルしたプログラムは、起動時にライブラリの署名を検査する。ライブラリが設置されていない状態でプログラムを実行すると、Syntax error となって停止する。これは故障ではなく、ローダーを先に実行していないことを示している。

コンパイラと BACONLDR.BIN は同じ世代のものを組み合わせること（1.2 節）。ローダーはリポジトリの

msx_basic_compiler/baconloader/ フォルダにあるものを使う。リポジトリ内のテスト用フォルダに残っている古いローダーは使わないこと。

2.3 MSX での実行手順

MSX の BASIC から、次の 2 行を実行する。

```
BLOAD "BACONLDR.BIN",R
BLOAD "HELLO.BIN",R
```

1. まず BACONLDR.BIN を「,R」付きで読み込む。「MSX-BACON Library installed.」と表示されればライブラリの設置は完了である。
2. 次にコンパイルしたバイナリを「,R」付きで読み込むと、そのまま実行が始まる。

プログラムが END 文に到達すると、実行は終了して BASIC のプロンプト（Ok）に戻る。

ディスクから自動で起動させたい場合は、次のような AUTOEXEC.BAS をディスクに置く方法が使える（付属のテストディスクもこの形式である）。

```
10 BLOAD "BACONLDR.BIN",R
20 BLOAD "HELLO.BIN",R
```

AUTOEXEC.BAS は MSX-BASIC が読める形式（改行は CR+LF）で用意すること。改行が LF だけのテキストでは Syntax error になる。

2.4 openMSX での実行

エミュレータ openMSX でも同じ手順で実行できる。ファイルを入れたフォルダをディスクとしてマウントする方法（DirAsDisk）が手軽である。

1. BACONLDR.BIN とコンパイルしたバイナリを同じフォルダに入れる。
2. そのフォルダをドライブ A としてマウントして openMSX を起動する。
3. BASIC から 2.3 節の 2 行を実行する。

本書の動作確認は、openMSX 22.0 上の Panasonic FS-A1ST（MSXturboR）で行っている。リポジトリ付属の PLAY テストディスク（test_play フォルダ）もこの方法で起動でき、「PASS PLAY ABI」と表示されることを確認している。

2.5 実行時のエラー

コンパイルしたプログラムの実行時エラーは、MSX-BASIC と同じエラーメッセージ（Illegal function call、Overflow、Out of memory など）で表示され、BASIC に戻る。

ただし、MSX-BACON は高速化のため実行時の検査を最小限に抑えている。MSX-BASIC ではエラーになる操作でも、検出されずにそのまま実行される場合がある。プログラムの正しさは、なるべくコンパイル時に決まる範囲で確保される前提で書く必要がある（3.12 節も参照）。

v0.9.2 からは ON ERROR GOTO によるエラー処理も書けるため、代表的な実行時エラーはプログラム側で受け止められる（第 4 章の ON ERROR GOTO の項）。ただし、エラーハンドラの中で発生したエラーはトラップできず、致命的になる。

2.6 メモリの使用

実行時に使われる主なメモリ領域は次のとおりである。

領域	内容
ページ 1（4000h 〜、約 9.2KB）	BACONLIB の常駐先（RAM）。実行中はスロットを切り替えて呼び出す
開始アドレス（既定 8010h）〜 HIMEM の下	コンパイルしたプログラム本体、変数、ヒープ、スタック
システムワーク F3D3h（BLIBSLOT）	ライブラリのスロット番号の保持に使う

コンパイルしたプログラムは HIMEM の下に配置され、スタックポインタは HIMEM に設定される。他の機械語プログラムと併用する場合は、開始アドレス（-start）や CLEAR 文で領域が重ならないように調整すること。

PLAY 命令を使うプログラムでは、プログラム内に演奏用のワーク領域 1152 バイトが追加で確保される。

第 3 章 MSX-BASIC での組み方の解説

この章では、MSX-BACON でコンパイルすることを前提に、MSX-BASIC プログラムをどう書くかを解説する。個々の命令・関数の詳細は第 4 章を参照されたい。

3.1 プログラムの形

- プログラムには行番号が必要である。行番号のない行はエラーになる。
- 1 行に複数の命令を書く場合は、MSX-BASIC と同様にコロン(:)で区切る。
- コメントは REM 文と「'」のどちらでも書ける。コンパイルしたプログラムの動作には影響しない。
- 飛び先には行番号のほか、ラベルも使える。ラベルは行の先頭に「*名前:」の形で定義し、「GOTO *名前」「GOSUB *名前」の形で参照する。ラベルは MSX-BACON 独自の書式で、MSX-BASIC にはない。

```
10 GOSUB *INIT
20 PRINT "MAIN"
30 END
100 *INIT:
110 PRINT "INIT"
120 RETURN
```

- プログラムの終わりには END 文を置く。END 文に到達すると BASIC に戻る。

3.2 変数と型

MSX-BACON の変数の型は MSX-BASIC と同じ 4 種類である。

型	識別子	内容
整数型	%	-32768 ～ 32767
単精度実数型	!	単精度浮動小数
倍精度実数型	#	倍精度浮動小数
文字列型	$	最大 255 文字の文字列

型	識別子	内容

型の決まり方には、MSX-BACON 独自の規則がある。

- 変数名の末尾に型識別子（%! # $）を付けた場合は、常にその型になる。
- 型識別子を省略した変数の型は、DEFINT / DEFSNG / DEFDBL / DEFSTR で決まる。ただし MSX-BASIC と違い、コンパイラはソースコード全体を走査して DEF 文を集め、頭文字ごとに「最後に現れた指定」を全体へ適用する。DEF 文より前に書かれた変数にも遡って効き、プログラムの途中で型が変わることはない。
- DEF 文で指定されていない頭文字の変数は、倍精度実数型になる（MSX-BASIC の既定と同じ）。
- 変数名の大文字・小文字は区別しない。
- -compatible モードでは、変数名は先頭 2 文字までが有効である（MSX-BASIC と同じ）。ABC と ABD は同じ変数になるので注意すること。-original モードでは名前全体が有効である。

```
10 DEFINT A-Z
20 COUNTER = 0
```

この例では、DEFINT A-Z により型識別子なしの変数はすべて整数型になる。

型の選択は速度に直結する。整数型の演算が最も速く、倍精度実数型が最も遅い。カウンタや座標には整数型、計算精度が必要な箇所にだけ実数型を使うのが基本である。

3.3 式と演算子

式の書き方は MSX-BASIC と同じである。使える演算子と優先順位は次のとおり（上が優先順位が高い）。

優先順位	演算子
高	^（べき乗）
	単項の + / -
	* /
	\（整数除算）

優先順位	演算子
	MOD
	+ -
	= <> >< > < >= => <= =< （比較）
	NOT
	AND
	OR
	XOR
	IMP
低	EQV

- 数値同士の演算では、両辺の型が倍精度 > 単精度 > 整数の順に揃えられてから計算される。
- 「+」は両辺が文字列なら文字列の連結になる。数値と文字列を混ぜると Type mismatch になる。
- 比較の結果は整数型で、真が -1、偽が 0 である。文字列同士の比較もできる。
- 論理演算子（NOT / AND / OR / XOR / EQV / IMP）は整数化した値に対するビット演算である。短絡評価は行われず、両辺とも常に評価される。
- 整数除算（\）と MOD は、両辺を整数に変換してから計算する。

なお、数値リテラルについては入力形式による違いが 1 点だけある。0.01 のような 0.1 未満の 10 進リテラルは、ASCII 形式で読み込んだ場合と中間言語形式で読み込んだ場合とで、内部の BCD イメージの正規化のされ方が異なる場合がある。数値としては同一であり、計算結果に影響はない。

3.4 制御構造

流れの制御には次の命令が使える（詳細は第 4 章）。

- FOR ～ NEXT による繰り返し。STEP の指定もできる。
- IF ～ THEN ～ ELSE による分岐。THEN の後は行番号、GOTO の省略形、命令のいずれも書ける。

- GOTO / GOSUB 〜 RETURN。飛び先は行番号でもラベルでもよい。
- ON 〜 GOTO / ON 〜 GOSUB による多分岐。
- ON ERROR GOTO によるエラー処理。ERL でエラーが起きた行番号、ERR でエラーコードが分かる。

ON ERROR GOTO / RESUME によるエラー処理も使える（v0.9.2 で対応）。ただし、エラーハンドラの中で発生したエラーはトラップできず致命的になるため、ハンドラは短く確実に書くこと。

プログラムの構造を素直に書くことが、最適化の面でも有利である。特に、ループの内側では型を統一し、不要な型変換が入らないようにすると速くなる。

3.5 文字列の扱い

- 文字列は最大 255 文字である。文字列変数同士の連結、切り出し（LEFT$ / MID$ / RIGHT$）、検索（INSTR）などが使える。
- MID$ 文による部分代入も使える。`A$="123456":MID$(A$,3)=A$` のような自己代入では、右辺を先に確定させてから置換する動作になる。
- SPC は、PRINT 文の中では空白の出力として、式の中では空白が並んだ文字列として使える（式中での利用は BACON 拡張で、インタプリタでは使えない）。TAB は PRINT 文専用のままで、式の中では使えない（ファイルへの PRINT # では TAB は何も出力しない）。
- INSTR は、開始位置を指定する 3 引数形も使える。INPUT$ も、キーボード入力形とファイル形（INPUT$(個数, #番号)）の両方が使える（3.8 節）。

3.6 配列

- 配列は DIM で宣言して使う。DIM A(10) は添字 0 〜 10 の 11 要素を確保する（MSX-BASIC と同じ）。OPTION BASE は仕様として使えない。コード生成全体が「添字は書いたとおり」を前提としており、添字の下限を切り替える仕組みは持たない。
- DIM なしで配列として使うと、暗黙に確保される経路もあるが、明示的に DIM することを勧める。確保済みの配列を再度 DIM するとエラーになる。
- 文字列操作の前に DIM を済ませておくこと。文字列のガベージコレクションで配列の配置が移動する場合があり、VARPTR で取得したアドレスがずれることがある（README の注意事項）。

3.7 DATA と READ

- DATA 文に置いた値を READ 文で順に読み取る。RESTORE で読み取り位置を戻せる。RESTORE の飛び先

には、行番号のほか *ラベルも使える。

- DATA の値の型は、読み取る変数の型に合わせて変換される。文字列配列への READ も使える。
- 数値データだけを機械語データとして配置したい場合は、BACON 独自の BINDATA / HEXDATA も使える（第 4 章）。

3.8 ファイル入出力

- OPEN でファイルを開き、PRINT # で書き込み、INPUT # / LINE INPUT # で読み取る。ランダムアクセスは FIELD / GET # / PUT # を使う。
- 対応するモードは OUTPUT / INPUT / ランダムアクセス / APPEND である（APPEND の実行時動作は v0.9.2 で修正済み）。
- INPUT # でファイルから読み取れる。ファイルの終わりを超えて読もうとすると実行時エラー 55（Input past end）になるため、EOF 関数で終わりを判定しながら読むのが基本である。
- LOC は現在位置を返す。シーケンシャルファイルでは、実際に消費したバイト数が返る。LOF はファイルの長さ、DSKF はドライブの空きクラスタ数を返す。
- GET # / PUT # はレコード番号を省略でき、その場合は現在位置のレコードが対象になり、位置が次へ進む。
- ファイル番号は #0 ～ #15 が使える。VARPTR(#n) で各ファイルの FCB のアドレスが得られる。

3.9 グラフィックとサウンド

- 画面モードは SCREEN 文で切り替える。PSET / PRESET / LINE / CIRCLE / PAINT、スプライトの PUT SPRITE、パレット操作などが使える。SCREEN 0 / 1 では POINT 関数がテキスト座標の文字コードを返す拡張がある。
- 効果音は BEEP と SOUND、音楽は PLAY 文が使える。PLAY は MSX-BASIC 標準の PSG 向け MML を演奏する。対応する MML は次の範囲である（詳細は第 4 章の PLAY の項）。

MML	内容
A ～ G（# / + / - で半音）	音符
R	休符
N0 ～ N96	音番号指定（N0 は休符）
O1 ～ O8	オクターブ

MML	内容
L1 ～ L64	既定の音長
T32 ～ T255	テンポ
V0 ～ V15	音量
M / S	エンベロープ周期 / 形状
X 変数$;	文字列変数の MML を展開
=変数;	数値パラメータを変数から取得
.（付点）	音符・休符の長さを 1.5 倍（複数可）

- MSX-MUSIC / MSX-AUDIO / MIDI 向けの拡張（< > Q @ や PLAY #機器指定）は対象外である。
- 演奏はバックグラウンドで進む。PLAY(n) 関数で演奏中かどうかを調べられる。

3.10 割り込み処理

次の割り込み系の命令が使える。

命令	起動条件
ON INTERVAL GOSUB	一定時間ごと
ON KEY GOSUB	ファンクションキーが押されたとき
ON SPRITE GOSUB	スプライトが衝突したとき
ON STOP GOSUB	CTRL+STOP が押されたとき
ON STRIG GOSUB	トリガーが押されたとき

それぞれ ON / OFF / STOP で有効・無効・保留を切り替える。割り込み処理の中で重い処理を行うと主処理に影響す

るため、手短に済ませること。

3.11 高速化の考え方

- 変数の型を意識する。ループ変数や座標は整数型（%）にし、DEFINT でまとめて宣言すると速くなる。
- 最適化レベルは既定の -O2 で十分である。定数式はコンパイル時に計算されるため、式を分解して書く必要はない。
- 実行時の検査が省かれている分、添字の範囲や型の一致は書き手が保証する必要がある（3.12 節）。

3.12 MSX-BASIC との相違点

MSX-BASIC のプログラムをそのままコンパイルできることが多いが、次の相違点がある。移植の際は特に確認すること。

相違点	内容
ソース編集系の命令は使えない（仕様）	AUTO、DELETE、EDIT、LIST、LLIST、RENUM、TRON / TROFF、NEW、CONT、CLOAD / CSAVE（プログラムのロード／セーブとしての意味）、MERGE など、対話・編集環境に対する命令は、コンパイル済みプログラムの中では意味を持たないため、仕様としてコンパイルできない（Syntax error になるのが正しい動作である）
実行時エラーは最小限	検出されないエラーがある。v0.9.2 からは ON ERROR GOTO / RESUME で代表的なエラーを受け止めることもできるが、ハンドラ内のエラーは致命的になる
DEF 文の効き方	記述位置ではなく、ソース全体から集めた指定が頭文字ごとに適用される（3.2 節）
変数名の有効長	-compatible では先頭 2 文字まで。異なるつもりの長い名前が衝突することがある
使えない命令・関数がある（仕様）	CALL 拡張の大半（対応は IOTGET / IOTPUT / IOTINIT / TURBO のみ）、PUT KANJI、SET VIDEO、SET PASSWORD、OPTION BASE などは仕様として非対応（第 4 章の対応状況一覧を参照）。ファイル関係の関数（EOF、LOC、LOF、DSKF など）と DEF FN は v0.9.2 で対応済み
一部関数の仕様差	FRE は引数の型によらず常に共通ヒープの空きを返す。CLEAR の第 1 引数（文字列領域サイズ）は受け付けるが無効である。どちらも、文字列と配列で共通ヒープを使う設計による仕様である。TAB は PRINT 専用のまま、LINE INPUT のキーボード形は全画面編集のない近似動作である

命令ごとの対応状況は、次章の対応状況一覧にまとめている。

第 4 章 対応命令リファレンス

この章では、MSX-BACON が対応している MSX-BASIC の命令・関数を解説する。まず対応状況一覧で全体の対応範囲を示し、続いて文と関数のリファレンスを載せる。各項目の「制限」は、実機の MSX-BASIC との違いである。

4.1 対応状況一覧

判定はコンパイラのソースコード（文・関数のハンドラの有無と実装内容）に基づく。「非対応」の命令・関数を書くと、コンパイル時に Syntax error になる（第 1 章 1.6 節）。MSX-BASIC の命令一覧（fu-sen 氏の MSX-BASIC リファレンス）を基準に照合した。

文は全 111 項目中、対応 78、部分対応 13、非対応 20。関数は全 74 項目中、対応 69、部分対応 5、非対応 0。CALL のサブコマンドは全 35 項目中、対応は BACON 独自の 4 項目（IOTGET / IOTINIT / IOTPUT / TURBO）のみで、残る 31 項目は仕様として非対応である。

なお、本書で「非対応」としているもののうち、次に挙げるものは未実装ではなく、仕様として対応しないことを決めているものである。これらを書いた場合は、Syntax error になるのが正しい動作である。

- 対話・編集系の命令：AUTO、DELETE、EDIT、LIST、LLIST、RENUM、TRON、TROFF、NEW、CONT、CLOAD / CSAVE（プログラムのロード／セーブとしての意味）、MERGE。コンパイル済みプログラムの中では意味を持たない。
- CALL 拡張コール：対応するのは CALL IOTGET / IOTPUT / IOTINIT / TURBO の 4 つだけである。漢字関係、MSX-MUSIC 関係など、その他の 31 項目は仕様として非対応である（4.1.3 節）。
- PUT KANJI、SET VIDEO、SET PASSWORD：仕様として非対応である。
- OPTION BASE：コード生成全体が「添字は書いたとおり」を前提とするため、仕様として非対応である（3.6 節）。

また、FRE と CLEAR の第 1 引数については、インタプリタとの違いを不具合ではなく仕様として定めている。BACON では文字列と配列が 1 つの共通ヒープを使うため、FRE は引数の型によらず常に共通ヒープの空きを返し、CLEAR の第 1 引数（文字列領域サイズ）は受け付けるが効果を持たない（各項を参照）。

4.1.1 文の対応状況

文	対応状況	備考
AUTO	非対応	対話・編集系のため仕様として非対応。Syntax error
BEEP	対応	—
BINDATA	対応	BACON 独自。0/1 文字列をバイト列にするデータ文
BLOAD	対応	R/S/オフセット全形式
BSAVE	対応	,S・実行アドレス含む
CALL	部分対応	IOTGET/IOTPUT/IOTINIT/TURBO のみ（他は仕様として非対応）。下記 CALL 表参照
CIRCLE	対応	STEP・全引数省略段階あり
CLEAR	部分対応	第 1 引数（文字列領域）は無効（共通ヒープ設計による仕様）、第 2 引数のみ実効
CLOAD	非対応	プログラムのロードはコンパイル済みプログラムでは意味を持たないため仕様として非対応。Syntax error
CLOSE	対応	リポジトリ内の対応表では「未実装」とされているが、実際は対応している
CLS	対応	—
COLOR	対応	通常形・=NEW・=RESTORE・=(p,r,g,b)・無引数
COLOR SPRITE	対応	$形も含む
CONT	非対応	対話・編集系のため仕様として非対応。Syntax error

文	対応状況	備考
COPY	部分対応	8 形式対応。COPY SCREEN 非対応、一部形式で方向指定無効
CSAVE	非対応	プログラムのセーブはコンパイル済みプログラムでは意味を持たないため仕様として非対応。Syntax error
DATA	対応	HEXDATA/BINDATA も同ハンドラで対応
DEF FN	対応	1 行定義のユーザー定義関数。定義は使用より前に書く
DEF USR	対応	DEFUSR[n]=式
DEFDBL	対応	プログラム全体を先行スキャンして適用する。出現位置に依存しない
DEFINT	対応	プログラム全体を先行スキャンして適用する。出現位置に依存しない
DEFSNG	対応	プログラム全体を先行スキャンして適用する。出現位置に依存しない
DEFSTR	対応	プログラム全体を先行スキャンして適用する。出現位置に依存しない
DELETE	非対応	対話・編集系のため仕様として非対応。Syntax error
DIM	対応	—
DRAW	対応	グラフィックマクロ言語。文字列の解釈はライブラリが行う
END	対応	—
ERASE	対応	再 ERASE でもエラーにならない差異あり
ERROR	対応	（エラー発生文として）
FIELD	部分	変数 16 個まで

文	対応状況	備考
	対応	
FILES	対応	—
FOR/NEXT	対応	/ 。NEXT の変数対応チェックは緩い
FRANDOMIZE	非対応	予約語のみでハンドラなし
GET	対応	GET # はレコード番号省略可（省略時は現在位置）
GET DATE	対応	,A オプション（アラーム日付）も v0.9.2 で対応
GET TIME	対応	GET DATE と同じく ,A オプションも v0.9.2 で対応
GOSUB	対応	行番号/*ラベルのみ
GOTO	対応	同上
HEXDATA	対応	BACON 独自。16 進文字列をバイト列にするデータ文
IF	対応	THEN/ELSE/GOTO、文・行番号・空すべて可
INPUT	部分対応	キーボード入力と INPUT #（ファイル入力）に対応。8 変数まで。ファイルの終端超過は実行時エラー 55
IRANDOMIZE	対応	BACON 独自の乱数初期化（RANDOMIZE の代替）
KEY	対応	（ON/OFF/LIST/割当て）＋（KEY(n) ON/OFF/STOP）
KILL	対応	—
LFILES	非対応	ハンドラなし

文	対応状況	備考
LINE	部分対応	始点側 STEP 不可。B/BF/LOP 対応
LINE INPUT	部分対応	キーボード形・ファイル形とも対応。キーボード形は全画面編集のない行入力近似
LIST	非対応	対話・編集系のため仕様として非対応。Syntax error
LLIST	非対応	対話・編集系のため仕様として非対応。Syntax error（LPRINT は対応）
LOAD	部分対応	が BLOAD の別名として受ける。プログラムロードの意味ではない
LOCATE	対応	—
LPRINT	対応	USING・SPC・TAB も処理
LSET	対応	—
MAXFILES	対応	の MAXFILES=<式> として処理（文として独立ハンドラはない）
MERGE	非対応	対話・編集系のため仕様として非対応。Syntax error
MID$ (文)	対応	長さ省略可
MOTOR	非対応	ハンドラなし。Syntax error
NAME	対応	—
NEW	非対	対話・編集系のため仕様として非対応。Syntax error

文	対応状況	備考
	応	
ON ERROR GOTO	対応	RESUME / ERL / ERR と組み合わせて使う。ハンドラ内のエラーは致命的
ON GOSUB	対応	—
ON GOTO	対応	飛び先省略可
ON INTERVAL GOSUB	対応	INTERVAL ON/OFF/STOP も同ハンドラ
ON KEY GOSUB	対応	飛び先最大 10
ON SPRITE GOSUB	対応	—
ON STOP GOSUB	対応	—
ON STRIG GOSUB	対応	飛び先最大 5
OPEN	対応	FOR INPUT/OUTPUT/APPEND・LEN・デバイス名可
OUT	対応	—
PAINT	部分対応	STEP 形不可
PLAY	対応	最大 3 チャンネル
POKES	対応	BACON 固有（文字列書き込み）
POKEW	対応	BACON 固有（2 バイト書き込み）
PRINT	対応	? 省略形も可
PRINT USING	対応	exec_using。PRINT # / LPRINT との併用可

文	対応状況	備考
PUT	対応	(PUT #)。レコード番号省略可(省略時は現在位置)
PUT KANJI	非対応	仕様として非対応。Syntax error
PUT SPRITE	部分対応	STEP 座標不可
RANDOMIZE	非対応	ハンドラなし。乱数初期化は IRANDOMIZE を使う
READ	対応	—
REM	対応	' も可
RENUM	非対応	対話・編集系のため仕様として非対応。Syntax error
RESTORE	対応	行番号・*ラベルとも可
RESUME	対応	RESUME NEXT / RESUME 行番号・*ラベルも可
RETURN	対応	RETURN 行番号/*ラベルも可
RSET	対応	(LSET と同一ハンドラ)
RUN	部分対応	RUN / RUN 行番号 / RUN "…"[,R] の形式は受けるが意味は再実行
SAVE	非対応	ハンドラなし。Syntax error
SCREEN	対応	全 6 引数

文	対応状況	備考
SET ADJUST	対応	SET ADJUSTS も可
SET BEEP	対応	—
SET DATE	対応	GET DATE と同じ形式の文字列で設定
SET PAGE	対応	—
SET PASSWORD	非対応	仕様として非対応。Syntax error
SET PROMPT	対応	—
SET SCREEN	対応	引数なし
SET SCROLL	対応	—
SET TIME	対応	GET TIME と同じ形式の文字列で設定
SET TITLE	対応	—
SET VIDEO	非対応	仕様として非対応。Syntax error
SOUND	対応	—
SPRITE ON/OFF/STOP	対応	が先頭 SPRITE を処理
STOP	部分対応	裸の STOP は END と同じ動作になる（CONT ができないための仕様）。STOP ON/OFF/STOP も可
SWAP	対応	区切りはカンマ

文	対応状況	備考
TIME (文)	対応	の TIME=<式> として処理
TROFF	非対応	対話・編集系のため仕様として非対応。Syntax error
TRON	非対応	対話・編集系のため仕様として非対応。Syntax error
VPOKES	対応	BACON 固有（VRAM へ文字列書き込み）
WAIT	対応	ポートの値が条件を満たすまで待機
WIDTH	部分対応	WIDTH LPRINT 形は非対応

4.1.2 関数の対応状況

関数	対応状況	備考
ABS	対応	絶対値。整数型の -32768 は倍精度へ昇格し 32768 を返す（実機と同じ）
ASC	対応	先頭文字のコード。空文字列時の厳密なエラー挙動はコード上保証なし
ATN	対応	arctan。引数は # 化して BIOS 演算
BASE	部分対応	VRAM ベースアドレス。定数表は実測に合わせて修正済み（SCREEN5/6 の SPGT は 7800h、SCREEN7 以上は F000h）。代入形 BASE(n)=v にも対応（BACON 拡張。インタプリタではエラー）。非定数の範囲外 ID では実機のような Illegal function call にならない場合がある
BIN$	対応	2 進表記。負数・大きな値は EXT% 経由
CDBL	対応	# への変換

関数	対応状況	備考
CHR$	対応	文字コード→1 文字
CHRHEX$	対応（固有）	**BACON 固有**。各文字を 2 桁 16 進コード列に展開。結果が長すぎると String too long（128 文字超の入力で結果 256 超）
CINT	対応	% 化。範囲外は実行時 Overflow（変換ルーチン依存）
COS	対応	cos
CSNG	対応	! への変換
CSRLIN	対応	カーソル行（Y）。読み取り専用システム変数的扱い
CVD	対応	先頭 8 文字を # のメモリイメージとして解釈。MKD$ の逆
CVI	対応	先頭 2 文字を % イメージとして解釈。MKI$ の逆
CVS	対応	先頭 **4 文字** を ! イメージとして解釈。MKS$ の逆。リポジトリ内の対応表の「先頭 8 文字」は誤り
DSKF	対応	ドライブの空きクラスタ数
EOF	対応	ファイル終端で -1
ERL	対応	エラー発生行番号。ON ERROR GOTO のハンドラで使う
ERR	対応	エラーコード。ON ERROR GOTO のハンドラで使う
EXP	対応	e^x
FIX	対応	0 方向へ丸め（小数部切捨て）。INT と違い負数でも切り上げ方向にならない

関数	対応状況	備考
FRE	部分対応	引数は評価されず、引数の型によらず常に共通ヒープ（文字列と配列で共用）の空きを返す（仕様）
FRND	対応（固有）	**BACON 拡張**。高速な実数乱数。インタプリタには関数として存在しない。状態は RND と共用
HEX$	対応	16 進表記
HEXCHR$	対応（固有）	**BACON 固有**。16 進文字列を 2 桁=1byte として文字列化。CHRHEX$ の逆。奇数桁・非 16 進文字の動作はライブラリ依存で未保証
INKEY$	対応	キーバッファ最古の 1 文字、無ければ ""
INP	対応	I/O ポート読み取り
INPUT$	対応	キーボード形とファイル形 INPUT$(n,#f) の両方に対応
INSTR	対応	部分文字列位置（先頭=1、無し=0、検索文字列 ""→1）。開始位置付き 3 引数形 INSTR(n,A$,B$) も対応
INT	対応	小数部切捨て（床方向。負数は実機同様より小さい整数側）
IRND	対応（固有）	**BACON 固有**。-32768～32767 の整数乱数。種は IRANDOMIZE 文（省略時 TIME）
LEFT$	対応	左から n 文字。n>長さなら全体、0→""。負／256 以上は不定（リポジトリ内の対応表）
LEN	対応	文字列長。数値変数では型サイズ（% は 2、! は 4、# は 8）を返す（BACON 拡張）
LOC	対応	現在位置。シーケンシャルファイルでは実際に消費したバイト数
LOF	対応	ファイルの長さ

関数	対応状況	備考
LOG	対応	自然対数
LPOS	対応	プリンタの現在位置（桁）

| MAX | 対応（固有） | **BACON 拡張**。引数の最大値（インタプリタには存在しない）。引数 2 個以上、戻り値の型は最も精度の高い引数の型 | | MID$（関数） | 対応 | 1 始まりで切り出し。開始>末尾→""、長さ 0→""。負・0・256 以上の開始／負・256 以上の長さは不定。代入形 `MID$(...)=` は別途 で対応（-compatible では実機の自己上書きバグまで再現、-original では通常動作） | | MKD$ | 対応 | # のメモリイメージ | | MKI$ | 対応 | % のメモリイメージ | | MKS$ | 対応 | ! のメモリイメージ | | MSXVER | 対応（固有） | **BACON 固有**（リポジトリ内の対応表も固有と明記）。ROM 002Dh を読み MSX1=0, MSX2=1, MSX2+=2, turboR=3 | | OCT$ | 対応 | 8 進表記 | | PAD | 対応 | BIOS GTPAD 呼び出し。リポジトリ内の対応表では番号 8〜19 のみ有効。それ以外は実機同様不定 | | PDL | 対応 | パドルのアナログ値（0〜255） | | PEEK | 対応 | RAM 1byte 読み | | PEEKS$ | 対応（固有） | **BACON 固有**。RAM からサイズ分を文字列化。サイズ 0〜255、それ以外は無保証（リポジトリ内の対応表） | | PEEKW | 対応（固有） | **BACON 固有**。RAM 2byte 読み（リトルエンディアン） | | PLAY | 対応 | PLAY 状態（チャンネル番号指定、演奏中判定）。PLAY 文とは別ハンドラ | | POINT | 対応 | 座標の色。BACON では SCREEN0/1 ではテキスト座標の文字コードを返す拡張あり（リポジトリ内の対応表） | | POS | 対応 | カーソルの現在位置（桁） | | RIGHT$ | 対応 | 右から n 文字。LEFT$ と同じ境界規則 | | RND | 部分対応 | BIOS RND 呼び出しで実機互換の乱数列。引数の符号による再シード等の意味は BIOS 依存。高速な FRND も使える（BACON 拡張）。括弧なし RND や引数省略形は不可 | | SGN | 対応 | -1/0/1 | | SIN | 対応 | sin | | SPACE$ | 対応 | スペース n 個 | | SPC | 対応 | PRINT 系の文の中では空白出力、式の中では空白の文字列として使用可（式中の利用は BACON 拡張） |

| SPRITE$ | 対応 | 代入形（パターン定義）と読み取り形の両方に対応 | | SQR | 対応 | 平方根。引数は # 化して BIOS 演算（定数引数はコンパイル時に計算）。配布された v0.9 では式ハンドラの登録漏れのため使用不可だったが、v0.9.1 で対応 | | STICK | 対応 | 0=カーソル,1/2=ジョイスティック。方向 0〜8 | | STR$ | 対応 | 数値→文字列（実機同様、正数は先頭スペース） | | STRIG | 対応 | 0=SPACE,1〜4=ジョイスティックボタン。押下で -1 | | STRING$ | 対応 | 第 2 引数が $ なら先頭文字、数値ならそのコードの文字を n 個 | | TAB | 部分対応 | PRINT 系の文の中でのみ使用可。ファイル出力時は何も出力しない。式の中では使えない | | TAN | 対応 | tan | | TIME | 対応 | JIFFY（0FC9Eh、1/60 秒カウンタ）の読み取り。代入 `TIME=式` も対応（ が EXT% で JIFFY へ書く） | | USR | 対応 | DEF USR で登録した機械語を呼ぶ。引数は VALTYP(F663h)/DAC(F7F6h) に実機形式で設定され型を機械語側で判別可。戻りも DAC 経由で

FRCINT/FRCSNG/FRCDBL 等で強制変換。詳細は本章の各項 | | VAL | 対応 | 文字列→数値。&H/&O/&B 形式も処理対象（ライブラリ依存） | | VARPTR | 対応 | 変数の値格納アドレス（変数名自体はコンパイル時に消えるが値の配置は実機互換）。VARPTR(#n) は #0=作業用 FCB、#1〜#15=各 FCB アドレス。範囲外ファイル番号は Illegal function call | | VDP | 部分対応 | VDP レジスタ読み。R# は書込専用のため最後に書いた値のバックアップを返す、S# 読み対応範囲はリポジトリ内の対応表参照（-9〜27 相当、範囲外は不定で実機のような Illegal function call にならない）。代入形 VDP(n)=式 は で対応（値は下位 8bit のみ有効、範囲外番号は何もしない） | | VPEEK | 対応 | VRAM 1byte 読み | | VPEEKS$ | 対応（固有） | **BACON 固有**。VRAM からサイズ分を文字列化。サイズ 0〜255、それ以外無保証 |

4.1.3 CALL サブコマンドの対応状況

CALL（省略形は _）で呼び出せるのは、IOTGET / IOTINIT / IOTPUT / TURBO の 4 つだけである。MSX-MUSIC や漢字 BASIC、MSX-DOS 系などの一般的な拡張コール（残る 31 項目）は、未実装ではなく仕様としてすべて非対応であり、書くと Syntax error になる。

サブコマンド	対応状況
CALL AKCNV	非対応
CALL ANK	非対応
CALL BGM	非対応
CALL CHDIR	非対応
CALL CHDRV	非対応
CALL CLS	非対応
CALL FORMAT	非対応
CALL IOTFIND	非対応（実装関数はあるが到達不能）
CALL IOTGET	対応（BACON 独自）
CALL IOTINIT	対応（BACON 独自）
CALL IOTPUT	対応（BACON 独自）

サブコマンド	対応状況
CALL JIS	非対応
CALL KACNV	非対応
CALL KANJI	非対応
CALL KEXT	非対応
CALL MFILES	非対応
CALL MKDIR	非対応
CALL MKILL	非対応
CALL MNAME	非対応
CALL MUSIC	非対応(実測)
CALL PALETTE	非対応
CALL PAUSE	非対応
CALL PCMPLAY	非対応
CALL PCMREC	非対応
CALL PITCH	非対応
CALL PLAY	非対応
CALL RAMDISK	非対応
CALL RMDIR	非対応

サブコマンド	対応状況
CALL SJIS	非対応
CALL STOPM	非対応
CALL SYSTEM	非対応
CALL TEMPER	非対応
CALL TRANSPOSE	非対応
CALL TURBO	対応（MSX ベーしっ君互換用。引数は無視される）
CALL VOICE	非対応

4.1.4 MSX-BACON 独自の命令・関数

MSX-BASIC にはない、MSX-BACON 独自の拡張は次のとおりである。

名前	種別	内容
*ラベル	文法	行番号の代わりに使う飛び先ラベル（3.1 節）
BINDATA	文	0/1 文字列をバイト列として配置するデータ文
HEXDATA	文	16 進文字列をバイト列として配置するデータ文
IRANDOMIZE	文	整数乱数の初期化（RANDOMIZE の代替）
POKEW	文	メモリへ 2 バイト書き込む
POKES	文	メモリへ文字列を書き込む
VPOKES	文	VRAM へ文字列を書き込む

名前	種別	内容
SET ADJUSTS	文	SET ADJUST の RTC へ書かない形
CALL IOTGET / IOTPUT / IOTINIT	文	MSX0 向けデバイス入出力
CHRHEX$	関数	16 進 2 桁ずつを文字コードとして文字列化する
HEXCHR$	関数	文字列を 16 進文字列化する
IRND	関数	整数乱数を返す
MSXVER	関数	MSX の世代（1〜4）を返す
PEEKW	関数	メモリから 2 バイト読む
PEEKS$	関数	メモリから文字列を読む
VPEEKS$	関数	VRAM から文字列を読む
USR の戻り値型指定	関数	USR(式) の後に型識別子を付けて戻り値の型を指定できる

| MAX | 関数 | 引数の最大値を返す（インタプリタには存在しない） | | FRND | 関数 | 高速な実数乱数を返す（インタプリタには関数として存在しない） | | BASE(n)= | 文 | VRAM ベースアドレスへの代入（インタプリタではエラーになる） | | LEN（数値変数） | 関数 | 数値変数の型サイズを返す（インタプリタではエラーになる） | | SPC（式中） | 関数 | 式の中で空白の文字列として使う（インタプリタでは PRINT 専用） |

4.2 文のリファレンス

対応・部分対応の文を、名前のアルファベット順に解説する。

4.2.1 BEEP

書式: BEEP ブザーを鳴らす。BIOS の BEEP ルーチンを呼び出すだけで、引数は取らない。

4.2.2 BINDATA

書式: BINDATA <2 進数データ>[, <2 進数データ> ...] BACON 固有のデータ文で、DATA と同じ位置にデータを配置する。各データは 0 と 1 だけで書き、8 文字ごとに 1 バイトへ変換されて文字列データになる。制限: データの文字数は 8 の倍数でなければならない。0/1 以外の文字を使うと Illegal function call になる。直後に ' でコメントを続けるとデータとして解釈されてエラーになるため、コメントは : で区切ること。例: `10 BINDATA 00111100,01111110`

4.2.3 BLOAD

書式: BLOAD <文字列式>[,R][, <オフセット>] / BLOAD <文字列式>, S[, <オフセット>] ファイル名で指定したバイナリファイルをメモリ(,R で即実行)または VRAM(,S)へ読み込む。オフセットを指定すると、ファイルに記録されたアドレスへその値を加算した位置に読み込む。制限: オフセットに変数 R は使えない(R が自動実行の指定と解釈されるため)。

4.2.4 BSAVE

書式: BSAVE <文字列式>, <開始アドレス>, <終了アドレス>[, <実行開始アドレス>] / BSAVE <文字列式>, <開始アドレス>, <終了アドレス>, S 開始アドレスから終了アドレスまでの RAM の内容をファイルへ書き出す。,S を付けると VRAM の内容を書き出す。実行開始アドレスを省略すると開始アドレスと同じになる。例: `10 BSAVE "HOGE.BIN", &H8000, &H8FFF`

4.2.5 CALL IOTGET

書式: CALL IOTGET(<デバイスパス文字列>, <変数名>) MSX0 において、デバイスパスで示されるデバイスから情報を取得し、指定した変数へ格納する。変数の型(整数・実数・文字列)はデバイスに合わせて選ぶこと。CALL の代わりに _ とも書ける。例: `10 CALL IOTGET("host/ip", IP$)`

4.2.6 CALL IOTINIT

書式：CALL IOTINIT() MSX0 において、ターミナル通信のためのフック初期化をやり直す。括弧は必須で、引数は書けない。

4.2.7 CALL IOTPUT

書式：CALL IOTPUT(<デバイスパス文字列>, <値>) MSX0 において、デバイスパスで示されるデバイスへ値を送信する。値は数値でも文字列でもよい。

4.2.8 CALL TURBO

書式：CALL TURBO ON|OFF [(<引数>)] MSX ベーしっ君のコンパイル制御命令との互換用で、BACON では読み捨てて無視する。ベーしっ君向けプログラムをそのままコンパイルできるようにするためのもので、実行時の動作はない。制限：括弧内は式の並びのみ可で、カンマを書くとエラーになる。

4.2.9 CIRCLE

書式：CIRCLE STEP, <半径>[, <色>[, <開始角>[, <終了角>[, <比率>]]]] 指定した中心と半径で円（弧）を描く。開始角・終了角・比率は単精度実数で指定し、後ろの引数から順に省略できる。例：`10 CIRCLE (100,100), 50, 15`

4.2.10 CLEAR

書式：CLEAR [<文字列領域サイズ>][, <上限アドレス>] 変数領域やスタックを初期化する。第 2 引数を指定すると HIMEM をその値に設定して初期化し直す。仕様：第 1 引数（文字列領域サイズ）は受け付けるが、効果はない。BACON では文字列と配列が共通ヒープを使うため、文字列領域だけを個別に指定する意味がないことによる。実質的に効くのは第 2 引数だけである。

4.2.11 CLOSE

書式：CLOSE [#<ファイル番号>[, #<ファイル番号> ...]] 開いているファイルを閉じる。ファイル番号を省略すると、開いている全てのファイルを閉じる。例：`10 CLOSE #1`

4.2.12 CLS

書式：CLS 画面を消去する。テキスト画面・グラフィック画面のどちらでも使える。

4.2.13 COLOR

書式: COLOR [<前景色>[, <背景色>[, <周辺色>]]] / COLOR [=NEW | =RESTORE | =(<パレット>, , ,)] 前景色・背景色・周辺色を設定する。引数を全て省略したり COLOR=NEW と書くとカラーパレットを初期化し、COLOR=RESTORE で VRAM 上のパレットへ戻す。COLOR=(...) の形ではパレット番号ごとに RGB（各 0〜7）を設定する。制限: 引数を途中だけ省略することはできない（COLOR 5, のような末尾カンマはエラー）。

4.2.14 COLOR SPRITE

書式: COLOR SPRITE(<プレーン番号>) = <色> / COLOR SPRITE$(<プレーン番号>) = <色文字列> スプライトプレーンの色を設定する。$ なしの形は 1 色を数値で、$ 付きの形は行ごとの色を文字列で指定する。

4.2.15 COPY

書式: COPY <転送元> TO <転送先> 画面領域・ファイル・配列変数の間で画像データを転送する。転送元と転送先には (X1,Y1)-(X2,Y2) の領域、ファイル名、配列変数名が使え、座標への転送ではページと論理演算（PSET, AND, OR, XOR, TPSET など）を指定できる。制限: COPY SCREEN は使えない。ファイルや配列が絡む形式では方向指定は無視される。例: `10 COPY (0,0)-(63,63), 0 TO (100,0), 1`

4.2.16 DATA

書式: DATA <定数>[, <定数> ...] READ 文で読むデータを配置する。BACON ではデータは全て文字列として保持され、READ 先の変数の型に合わせて変換される。制限: 引用符なしのデータに : を含めることはできない（文の区切りと解釈される）。時刻などは "00:00:00" のように引用符で囲むこと。例: `10 DATA 1, 2, "ABC"`

4.2.17 DEF FN

書式: DEF FN<名前>(<仮引数>) = <式> 1 行で書くユーザー定義関数を定義する。数値関数・文字列関数のどちらも定義でき、定義した関数は FN<名前>(<引数>) の形で式の中から呼び出す。制限: 定義は使用より前に書かなければならない（インタプリタと同じ）。複数行にわたる定義はできない。例: `10 DEF FNSQ(X)=X*X: PRINT FNSQ(3)`

4.2.18 DEF USR

書式: DEFUSR[<番号>] = <開始アドレス> USR 関数で呼び出す機械語ルーチンの開始アドレスを登録する。番号は 0〜9 の 1 桁で、省略すると 0 になる。例: `10 DEFUSR0 = &HC000`

4.2.19 DEFDBL

書式：DEFDBL <文字>[-<文字>][, ...] 型接尾辞を省略した変数のうち、指定した頭文字のものを倍精度実数型として扱う。プログラム全体を先行して読み取り、出現位置によらず全体へ適用される。制限：プログラムの途中で型を切り替える使い方はできない。同じ文字への再指定は最後のものが有効。

4.2.20 DEFINT

書式：DEFINT <文字>[-<文字>][, ...] 型接尾辞を省略した変数のうち、指定した頭文字のものを整数型として扱う。プログラム全体を先行して読み取り、出現位置によらず全体へ適用される。制限：プログラムの途中で型を切り替える使い方はできない。同じ文字への再指定は最後のものが有効。例：`10 DEFINT A-Z`

4.2.21 DEFSNG

書式：DEFSNG <文字>[-<文字>][, ...] 型接尾辞を省略した変数のうち、指定した頭文字のものを単精度実数型として扱う。プログラム全体を先行して読み取り、出現位置によらず全体へ適用される。制限：プログラムの途中で型を切り替える使い方はできない。同じ文字への再指定は最後のものが有効。

4.2.22 DEFSTR

書式：DEFSTR <文字>[-<文字>][, ...] 型接尾辞を省略した変数のうち、指定した頭文字のものを文字列型として扱う。プログラム全体を先行して読み取り、出現位置によらず全体へ適用される。制限：プログラムの途中で型を切り替える使い方はできない。同じ文字への再指定は最後のものが有効。

4.2.23 DIM

書式：DIM <配列変数>(<要素数>[, <要素数> ...])[, ...] 配列変数を宣言し、メモリを確保する。要素数に指定した値まで（0 からその値まで）の要素が使えるようになる。例：`10 DIM A(10), B$(5, 5)`

4.2.24 DRAW

書式：DRAW <文字列式> グラフィックマクロ言語で図形を描く。文字列中の命令（U / D / L / R などの移動と描画、M の座標指定、B / N の前置指定、C の色、S の倍率、A の回転、X の文字列展開など）に従って描画する。文字列の解釈は BACONLIB が実行時に行う。例：`10 DRAW "BM100,100 C15 U50 R50 D50 L50"`

4.2.25 END

書式：END プログラムの実行を終了する。終了後は BASIC へ戻る処理へジャンプする。

4.2.26 ERASE

書式：ERASE <配列変数名>[, <配列変数名> ...] 指定した配列変数のメモリを解放する。解放した配列は DIM で再宣言できる。制限：MSX-BASIC では解放済みの配列を指定するとエラーになるが、BACON ではエラーにならず何も起きない。

4.2.27 ERROR

書式：ERROR <エラー番号> 指定した番号のエラーを発生させる。エラー処理のテストなどに使う。制限：ON ERROR GOTO でトラップしている場合はハンドラへ制御が移る。トラップしていない場合は、そのままプログラム終了につながる。

4.2.28 FIELD

書式：FIELD #<ファイル番号>, <長さ> AS <文字列変数>[, <長さ> AS <文字列変数> ...] ランダムアクセスファイルに対して、レコード内の各項目の長さと対応する文字列変数を割り当てる。GET # / PUT # と組み合わせて使う。制限：1 つの FIELD 文で指定できる変数は最大 16 個まで（17 個以上はエラー）。変数は文字列のスカラ変数でなければならない。例：`10 FIELD #1, 10 AS A$, 20 AS B$`

4.2.29 FILES

書式：FILES [<ワイルドカード文字列>] ディスク上のファイルの一覧を表示する。ワイルドカード文字列を省略すると全ファイルが対象になる。例：`10 FILES "*.BAS"`

4.2.30 FOR / NEXT

書式：FOR <変数> = <初期値> TO <終値> [STEP <増分>] ... NEXT [<変数>] ループ変数を初期値から終値まで増分ずつ変化させながら、FOR と NEXT の間を繰り返す。NEXT の変数は省略でき、複数の変数をカンマで並べることもできる。例：`10 FOR I = 1 TO 10 STEP 2: PRINT I: NEXT I`

4.2.31 GET

書式：GET #<ファイル番号>[, <レコード番号>] ランダムアクセスファイルから指定レコードを読み、FIELD で割り当てた変数へ入れる。レコード番号は先頭が 1 である。レコード番号を省略すると現在位置のレコードが対象になり、読み取り後に位置が次へ進む。例：`10 GET #1, 3`

4.2.32 GET DATE

書式：GET DATE <文字列変数> 現在の日付を文字列として取得し、文字列変数へ格納する。書式は MSX のリージョン設定に従う。制限：アラーム日付を取得する ,A オプションも使える（v0.9.1 以前はパーサの不具合で Syntax error になっていたが、v0.9.2 で修正された）。

4.2.33 GET TIME

書式：GET TIME <文字列変数> 現在の時刻を「時:分:秒」形式の文字列として取得し、文字列変数へ格納する。制限：アラーム時刻を取得する ,A オプションも使える（GET DATE と同じく v0.9.2 で修正された）。

4.2.34 GOSUB

書式：GOSUB <行番号> / GOSUB *<ラベル> 指定した行番号またはラベルへサブルーチンコールする。RETURN で呼び出し元へ戻る。例：`10 GOSUB 1000`

4.2.35 GOTO

書式：GOTO <行番号> / GOTO *<ラベル> 指定した行番号またはラベルへ無条件にジャンプする。飛び先に式は使えない。例：`10 GOTO *LOOP`

4.2.36 HEXDATA

書式：HEXDATA <16 進数データ>[, <16 進数データ> ...] BACON 固有のデータ文で、DATA と同じ位置にデータを配置する。各データは 16 進文字列として解釈され、2 桁ごとに 1 バイトへ変換されて文字列データになる。制限：データの文字数は偶数でなければならない。16 進数として不正な文字を使うと Illegal function call になる。例：`10 HEXDATA 3132, 4142`

4.2.37 IF

書式：IF <条件式> THEN {<文> | <行番号>} [ELSE {<文> | <行番号>}] / IF <条件式> GOTO <行番号> 条件式が 0 でなければ THEN 側を、0 なら ELSE 側を実行する。THEN/GOTO の後には文でも行番号でも書け、THEN で行を終える（何もしない）形も許される。制限：条件式に文字列は使えない（Type mismatch になる）。

4.2.38 INPUT

書式：INPUT ["<プロンプト>";] <変数>[, <変数> ...] / INPUT #<ファイル番号>, <変数>[, <変数> ...] キーボードから入力された値を変数へ格納する。プロンプト文字列はリテラルで書き、区切りは ; でなければならない。INPUT #

の形では、シーケンシャルファイルから値を読み取る。制限：変数は最大 8 個まで（9 個以上は Overflow エラー）。ファイル入力で終端を超えて読もうとすると、実行時エラー 55（Input past end）になる。例：`10 INPUT "NAME"; N$`

4.2.39 IRANDOMIZE

書式：IRANDOMIZE [<シード値>] BACON 固有の文で、整数乱数 IRND() のシードを初期化する。シード値を省略すると TIME の値が使われる。

4.2.40 KEY

書式：KEY ON / KEY OFF / KEY LIST / KEY <キー番号>, <文字列> ファンクションキーの表示（ON）、消去（OFF）、割当て内容の一覧表示（LIST）を行う。番号と文字列を指定する形では、そのファンクションキーへ文字列を割り当てる。例：`10 KEY 1, "RUN" + CHR$(13)`

4.2.41 KEY(n) ON/OFF/STOP

書式：KEY(<キー番号>) ON / KEY(<キー番号>) OFF / KEY(<キー番号>) STOP ON KEY GOSUB によるファンクションキー割り込みを、キーごとに有効（ON）・無効（OFF）・保留（STOP）にする。

4.2.42 KILL

書式：KILL <ワイルドカード文字列> ワイルドカードにマッチするファイルを削除する。例：`10 KILL "*.BAK"`

4.2.43 LET

書式：[LET] <変数> = <式> 変数へ式の評価結果を代入する。LET は省略できるのが普通で、配列要素への代入も同じ形で書ける。

4.2.44 LINE

書式：LINE [(,)]-STEP[, <色>[, {B | BF}[, <論理演算>]]] 2 点間に線を引く。B を付けると矩形の枠、BF を付けると塗りつぶした矩形になる。論理演算には PSET, PRESET, AND, OR, XOR, TPSET などが指定できる。制限：始点側に STEP は付けられない（STEP が使えるのは終点側のみ）。例：`10 LINE (10,10)-(100,50), 15, BF`

4.2.45 LINE INPUT

書式：LINE INPUT ["<プロンプト>";] <文字列変数> / LINE INPUT #<ファイル番号>, <文字列変数> キーボードまたはファイルから 1 行分を読み取り、文字列変数へ格納する。カンマや前後の空白もそのまま取り込まれる点が

INPUT 文と異なる。制限：キーボード形では、インタプリタのような全画面編集はできない（行入力の近似動作になる）。例：`10 LINE INPUT "NAME"; N$`

4.2.46 LOAD

書式：LOAD <文字列式>[, R][, <オフセット>] BLOAD と同じ処理を行う別名で、バイナリファイルを読み込む。プログラムファイルを読み込んで実行する LOAD ではない。制限：BLOAD と異なり ,S 形式と ,R の後にオフセットを続ける形式は使えない。

4.2.47 LOCATE

書式：LOCATE [][,][, <カーソルスイッチ>] テキスト画面のカーソル位置を移動する。各引数は個別に省略でき、カーソルスイッチは 0 で非表示、それ以外で表示になる。制限：引数を 1 つも書かない書き方や、末尾をカンマで終える書き方はエラーになる場合がある。

4.2.48 LPRINT

書式：LPRINT [<式> ...] / LPRINT USING <書式>; <引数> ... PRINT と同じ書式で、出力先がプリンタになる。; による改行抑止、, によるゾーン区切り、SPC()、TAB()、USING が使える。注記：v0.9.1 以前は PTRFIL の設定不具合のため、LPRINT の出力がプリンタへ届かなかった。v0.9.2 で修正された。例：`10 LPRINT "TITLE"; A$`

4.2.49 LSET

書式：LSET <文字列変数> = <文字列式> 文字列変数の現在の長さに合わせて、式の結果を左詰めで入れる。短い場合は右にスペースが詰められ、長い場合ははみ出した分が切り捨てられる。FIELD で割り当てた変数への書き込みに使う。

4.2.50 MAXFILES

書式：MAXFILES = <式> 同時に開けるファイルの上限数を設定する。代入文の形で書く。例：`10 MAXFILES = 2`

4.2.51 MID$

書式：MID$(<文字列変数>, <開始位置>[, <長さ>]) = <文字列式> 文字列変数の一部を、指定した文字列で置き換える。長さを省略すると、代入する文字列の長さ分だけが置き換わる。例：`10 MID$(A$, 3, 2) = "XY"` 制限：右辺に置換対象と同じ変数を使う自己代入（例：MID$(A$,3)=A$）では、右辺を先に確定させてから置換する動作になる。コンパイルモードによる違いはない（著者の対応表にはモードで変わる旨の記述があるが、現行コードには分岐がない）

【要確認】

4.2.52 NAME

書式：NAME <旧ファイル名> AS <新ファイル名> ファイル名を変更する。どちらの名前にもワイルドカードが使える。
例：`10 NAME "OLD.BAS" AS "NEW.BAS"`

4.2.53 ON ERROR GOTO

書式：ON ERROR GOTO <行番号> / ON ERROR GOTO *<ラベル> / ON ERROR GOTO 0 実行時エラーが発生したときに、指定した行のエラーハンドラへ制御を移す。0 を指定するとトラップを無効にする。ハンドラの中では、ERL でエラーの発生行、ERR でエラーコードが分かり、RESUME で実行を再開する。制限：ハンドラの中で発生したエラーはトラップできず、致命的なエラーとなってプログラムが終了する。例：`10 ON ERROR GOTO 1000`

4.2.54 ON GOTO / ON GOSUB

書式：ON <式> GOTO <飛び先>[, <飛び先> ...] / ON <式> GOSUB <飛び先>[, <飛び先> ...] 式の値が 1 なら 1 番目の飛び先、2 なら 2 番目の飛び先へ分岐する。値が 0 または飛び先の個数を超える場合は分岐せず次の文へ進む。途中の飛び先は ,, と省略できる。例：`10 ON A GOTO 100, 200, 300`

4.2.55 ON INTERVAL GOSUB

書式：ON INTERVAL = <タイマー値> GOSUB <行番号> タイマー割り込みの間隔と、割り込み時に呼ぶサブルーチンを指定する。タイマー値は 1/60 秒（PAL では 1/50 秒）単位である。INTERVAL ON/OFF/STOP で割り込みの有効・保留を切り替える。例：`10 ON INTERVAL = 60 GOSUB 1000: INTERVAL ON`

4.2.56 ON KEY GOSUB

書式：ON KEY GOSUB <行番号>[, <行番号> ...] ファンクションキーが押されたときに呼ぶサブルーチンを、F1〜F10 の順に指定する。実際に割り込みが起きるのは KEY(n) ON で有効にしたキーだけである。制限：飛び先は最大 10 個まで。途中の飛び先の省略はできない。

4.2.57 ON SPRITE GOSUB

書式：ON SPRITE GOSUB <行番号> スプライト同士が衝突したときに呼ぶサブルーチンを指定する。SPRITE ON/OFF/STOP で割り込みの有効・保留を切り替える。

4.2.58 ON STOP GOSUB

書式：ON STOP GOSUB <行番号> CTRL+STOP が押されたときに呼ぶサブルーチンを指定する。STOP ON/OFF/STOP で割り込みの有効・保留を切り替える。

4.2.59 ON STRIG GOSUB

書式：ON STRIG GOSUB <行番号>[, <行番号> ...] トリガボタンが押されたときに呼ぶサブルーチンを、スペースキー、ジョイスティック 1/2 のボタン順に指定する。STRIG(n) ON/OFF/STOP で割り込みの有効・保留を切り替える。
制限：飛び先は最大 5 個まで。

4.2.60 OPEN

書式：OPEN <ファイル名> [FOR {INPUT | OUTPUT | APPEND}] AS #<ファイル番号> [LEN = <レコード長>] ファイルやデバイスを開く。FOR を省略するとランダムアクセスモードになり、その場合だけ LEN でレコード長を指定できる。"GRP:" や "CAS:" などのデバイス名もファイル名として書ける。例：`10 OPEN "DATA.DAT" AS #1 LEN = 128`

4.2.61 OUT

書式：OUT *, <値> 指定した I/O ポートへ値を出力する。*

4.2.62 PAINT

書式：PAINT (,)[, <塗り色>[, <境界色>]] 指定座標を含む領域を塗り色で塗りつぶす。境界色を省略すると塗り色と同じ色が境界になる。制限：座標に STEP は付けられない。例：`10 PAINT (100,100), 15, 8`

4.2.63 PLAY

書式：PLAY [<文字列式 1>][, <文字列式 2>][, <文字列式 3>]] 各チャンネルの MML 文字列を演奏する。チャンネルごとの文字列は省略できる。制限：チャンネルは最大 3 つまで（4 つ以上書くとエラー）。

4.2.64 POKE

書式：POKE <アドレス>, <値> メモリの指定アドレスへ 1 バイトの値を書き込む。

4.2.65 POKES

書式：POKES <アドレス>, <文字列式> BACON 固有の文で、メモリの指定アドレスへ文字列の内容をそのまま書き

込む。

4.2.66 POKEW

書式：POKEW <アドレス>, <値> BACON 固有の文で、メモリの指定アドレスへ 2 バイトの値を書き込む。

4.2.67 PRINT

書式：PRINT [<式> ...] / ? [<式> ...] 式の値を画面へ出力する。; で区切ると続けて出力し、, で区切ると次のゾーン位置から出力する。SPC() と TAB() が使え、末尾の ; で改行を抑止する。? は PRINT の省略形である。

4.2.68 PRINT

書式：PRINT #<ファイル番号>, [<式> ...] PRINT と同じ書式で、開いているファイルへ出力する。制限：ファイル出力では TAB() は何も出力しない。

4.2.69 PRINT USING

書式：PRINT USING <書式文字列>; <引数>[, <引数> ...] 書式文字列に従って引数を整形して出力する。書式と引数の区切りは ; でなければならない。PRINT # や LPRINT と組み合わせることもできる。制限：引数の合計サイズが内部バッファ（258 バイト相当）を超えると Illegal function call になる。例：`10 PRINT USING "#.#"; 1.5`

4.2.70 PSET / PRESET

書式：PSET (,)[, <色>[, <論理演算>]] / PRESET (,)[, <色>[, <論理演算>]] 指定座標に点を描く。色を省略すると PSET は前景色、PRESET は背景色になる。論理演算には PSET, AND, OR, XOR, TPSET などが指定できる。制限：座標に STEP は付けられない。例：`10 PSET（100，100），15`

4.2.71 PUT

書式：PUT #<ファイル番号>[, <レコード番号>] FIELD で割り当てた変数の内容を、ランダムアクセスファイルの指定レコードへ書き込む。レコード番号は先頭が 1 である。レコード番号を省略すると現在位置のレコードが対象になり、書き込み後に位置が次へ進む。例：`10 PUT #1，3`

4.2.72 PUT SPRITE

書式：PUT SPRITE <プレーン番号>, [(,)][, <色>][, <パターン番号>] スプライトプレーンの座標・色・パターンを設定する。座標、色、パターンのうち少なくとも 1 つは指定しなければならず、後ろのものから省略できる。制限：座標

に STEP は付けられない。例：10 PUT SPRITE 0，(100，100)，15，0

4.2.73 READ

書式：READ <変数>[, <変数> ...] DATA 文のデータを現在の参照位置から順に読み、変数へ格納する。データは文字列として保持されているため、変数の型に合わせて変換される。

4.2.74 REM

書式：REM <コメント> / ' <コメント> コメントを書く。コードは生成されず、コンパイル結果のアセンブリへコメントとして残る。文の途中から : を介して書くこともできる。

4.2.75 RESTORE

*書式：RESTORE [<行番号> | *<ラベル>] READ の参照位置を指定行番号以降で最初の DATA 文へ戻す。行番号を省略するとプログラムの最初の DATA 文が対象になる。飛び先には行番号のほか *ラベルも使える。*

4.2.76 RESUME

*書式：RESUME [<行番号> | *<ラベル>] / RESUME NEXT ON ERROR GOTO のエラーハンドラから実行を再開する。引数なしではエラーが発生した文から、RESUME NEXT ではエラーの次の文から、行番号・ラベルを指定した場合はその行から再開する。制限：エラーハンドラの外では使えない。ハンドラ内でのエラーは致命的になる（ON ERROR GOTO の項参照）。*

4.2.77 RETURN

*書式：RETURN [<行番号> | *<ラベル>] GOSUB で呼ばれたサブルーチンから戻る。行番号やラベルを指定すると、戻りアドレスを破棄してその行へジャンプする。*

4.2.78 RSET

書式：RSET <文字列変数> = <文字列式> 文字列変数の現在の長さに合わせて、式の結果を右詰めで入れる。短い場合は左にスペースが詰められ、長い場合ははみ出した分が切り捨てられる。FIELD で割り当てた変数への書き込みに使う。

4.2.79 RUN

書式：RUN / RUN <行番号> / RUN <文字列式>[, R] コンパイルされたプログラム自身を先頭（または指定行）から

実行し直す。ファイル名を指定する形も書ける。制限：MSX-BASIC のように別プログラムをロードして実行するものではない。

4.2.80 SCREEN

書式：SCREEN [<モード>[, <スプライトサイズ>[, <キークリック>[, <ボーレート>[, <プリンタ種別>[, <インタレース>]]]]] 画面モードと付随する設定を変更する。引数は後ろから順に省略できる。制限：カンマだけを残して途中の引数を省略することはできない（Missing operand になる）。例：`10 SCREEN 5`

4.2.81 SET ADJUST / SET ADJUSTS

書式：SET ADJUST (,) / SET ADJUSTS (,) 画面の表示位置を調整する。SET ADJUST は設定を RTC へ書き込み、SET ADJUSTS は書き込まない。

4.2.82 SET BEEP

書式：SET BEEP [<音色>][, <音量>] BEEP 音の音色と音量を設定する。どちらか一方だけの指定もできる。制限：引数を 1 つも書かないとエラーになる。

4.2.83 SET DATE / SET TIME

書式：SET DATE <文字列式> / SET TIME <文字列式> 現在の日付・時刻を設定する。文字列の形式は、GET DATE / GET TIME で得られる形式と同じである。例：`10 SET TIME "12:34:56"`

4.2.84 SET PAGE

書式：SET PAGE [<表示ページ>][, <描画ページ>] SCREEN 5 以降で、表示ページと描画ページを指定する。どちらか一方だけの指定もできる。制限：引数を 1 つも書かないとエラーになる。例：`10 SET PAGE 1, 0`

4.2.85 SET PROMPT

書式：SET PROMPT <文字列式> BASIC のプロンプト文字列を設定する。文字列式は必須である。

4.2.86 SET SCREEN

書式：SET SCREEN 現在の画面モードなどの設定を、起動時のデフォルトとして記憶する。引数は取らない。

4.2.87 SET SCROLL

書式：SET SCROLL [][,][, <マスクモード>][, <ページモード>] ハードウェアスクロールの位置とモードを設定する。各引数は個別に省略できる。制限：引数を 1 つも書かないとエラーになる。

4.2.88 SET TITLE

書式：SET TITLE <文字列式>[, <色>] 起動画面に表示するタイトルとその色を設定する。

4.2.89 SOUND

書式：SOUND , <値> PSG の指定レジスタへ値を書き込む。

4.2.90 SPRITE ON/OFF/STOP

書式：SPRITE ON / SPRITE OFF / SPRITE STOP ON SPRITE GOSUB によるスプライト衝突割り込みを有効（ON）・無効（OFF）・保留（STOP）にする。

4.2.91 SPRITE$ 代入

書式：SPRITE$(<パターン番号>) = <文字列式> スプライトパターンを文字列で定義する（代入文として書く）。文字列はパターンの大きさに応じた長さが必要である。

4.2.92 STOP ON/OFF/STOP

書式：STOP ON / STOP OFF / STOP STOP ON STOP GOSUB による CTRL+STOP 割り込みを有効（ON）・無効（OFF）・保留（STOP）にする。裸の STOP 文も書ける。コンパイルされたプログラムでは継続実行（CONT）はできないため、裸の STOP は END と同じく BASIC へ戻る動作になる（仕様）。

4.2.93 STRIG(n) ON/OFF/STOP

書式：STRIG(<番号>) ON / STRIG(<番号>) OFF / STRIG(<番号>) STOP ON STRIG GOSUB によるトリガ割り込みを、ボタンごとに有効（ON）・無効（OFF）・保留（STOP）にする。

4.2.94 SWAP

書式：SWAP <変数 1>, <変数 2> 2 つの変数の内容を入れ替える。2 つの変数は同じ型でなければならない。制限：区切りはカンマである（= では書けない）。型が異なると Type mismatch になる。

4.2.95 TIME =

書式：TIME = <式> システムタイマへ値を代入する（代入文として書く）。

4.2.96 VDP(n) =

書式：VDP(<レジスタ番号>) = <式> VDP のレジスタへ値を書き込む（代入文として書く）。

4.2.97 VPOKE

書式：VPOKE <アドレス>, <値> VRAM の指定アドレスへ 1 バイトの値を書き込む。

4.2.98 VPOKES

書式：VPOKES <アドレス>, <文字列式> BACON 固有の文で、VRAM の指定アドレスへ文字列の内容をそのまま書き込む。

4.2.99 WAIT

書式：WAIT <ポート番号>, <マスク>[, <反転値>] 指定した I/O ポートを繰り返し読み、(読み取った値 XOR <反転値>)AND <マスク> が 0 でなくなるまで待機する。<反転値>を省略すると 0 として扱われる。

4.2.100 WIDTH

書式：WIDTH <式> テキスト画面の 1 行の文字数を変更する。制限：WIDTH LPRINT <式> の形は使えない。例：

```
10 WIDTH 40
```

4.3 関数のリファレンス

対応・部分対応の関数を、名前のアルファベット順に解説する。

4.3.1 ABS

書式：ABS(<数値>) 引数<数値>の絶対値を返します。通常、戻り値の型は引数と同じ型になります。制限：整数型の -32768 は、絶対値 32768 が整数型に収まらないため、倍精度実数へ型が上がって返ります（インタプリタと同じ動作）。

4.3.2 ASC

書式：ASC(<文字列>) 文字列の先頭文字の文字コード（ASCII コード）を整数で返します。

4.3.3 ATN

書式：ATN(<値>) <値>のアークタンジェント（逆正接）をラジアンで返します。戻り値は倍精度実数です。

4.3.4 BASE

書式：BASE( ) / BASE( ) = <値> に対応する VRAM 上のベースアドレスを返します。代入形ではベースアドレスを設定します。代入形は MSX-BACON 独自の拡張で、インタプリタではエラーになります。制限：対応しない ID や範囲外の値でもエラーにならず、意味のない値を返すことがあります。

4.3.5 BIN$

書式：BIN$(<値>) <値>を 2 進数表記の文字列に変換して返します。

4.3.6 CDBL

書式：CDBL(<値>) 整数・単精度実数・倍精度実数の<値>を倍精度実数に変換して返します。戻り値は必ず倍精度実数型になります。

4.3.7 CHR$

書式：CHR$(<文字コード>) <文字コード>に対応する 1 文字の文字列を返します。

4.3.8 CHRHEX$

書式：CHRHEX$(<文字列>) 文字列の各文字を、1 文字あたり 2 桁の 16 進数で表した文字コード列の文字列に変換して返します。MSX-BACON 独自の関数です。制限：結果が 255 文字を超えるような長い文字列は渡せません。

4.3.9 CINT

書式：CINT(<値>) <値>を整数に変換して返します。戻り値は必ず整数型になります。制限：変換結果が -32768～32767 に収まらない場合は Overflow エラーになります。

4.3.10 COS

書式：COS(<値>) <値>（ラジアン）のコサインを返します。戻り値は倍精度実数です。

4.3.11 CSNG

書式：CSNG(<値>) 整数・単精度実数・倍精度実数の<値>を単精度実数に変換して返します。戻り値は必ず単精度実数型になります。

4.3.12 CSRLIN

書式：CSRLIN 現在のカーソルの行位置（Y 座標）を返します。括弧は付けません。読み取り専用です。

4.3.13 CVD

書式：CVD(<文字列>) 文字列の先頭 8 文字を、倍精度実数のメモリイメージとして数値に変換して返します。MKD$ の逆変換です。

4.3.14 CVI

書式：CVI(<文字列>) 文字列の先頭 2 文字を、整数のメモリイメージとして数値に変換して返します。MKI$ の逆変換です。

4.3.15 CVS

書式：CVS(<文字列>) 文字列の先頭 4 文字を、単精度実数のメモリイメージとして数値に変換して返します。MKS$ の逆変換です。

4.3.16 DSKF

書式：DSKF(<ドライブ番号>) 指定ドライブの空き容量をクラスタ数で返します。<ドライブ番号>は、0 がカレントドライブ、1 が A ドライブ、2 が B ドライブです。

4.3.17 EOF

書式：EOF(<ファイル番号>) シーケンシャル入力ファイルが終端に達していれば -1 を、そうでなければ 0 を返します。INPUT # で読み取る前に、終端を判定するために使います。

4.3.18 ERL

書式：ERL エラーが発生した行番号を返します。括弧は付けません。ON ERROR GOTO のエラーハンドラの中で使います。

4.3.19 ERR

書式：ERR 発生したエラーのエラーコードを返します。括弧は付けません。ON ERROR GOTO のエラーハンドラの中で使います。

4.3.20 EXP

書式：EXP(<値>) 自然対数の底 e の <値> 乗、exp(<値>) を返します。戻り値は倍精度実数です。

4.3.21 FIX

書式：FIX(<値>) <値>の小数部を切り捨て、整数部分だけを返します。戻り値の型は引数と同じです。負の値でも 0 に近い側へ丸めます。

4.3.22 FRE

書式：FRE(<式>) ヒープメモリの空き容量を返します。仕様：<式>は評価されません。インタプリタは引数が数値か文字列かで返り値を区別しますが、BACON では文字列と配列が 1 つの共通ヒープを使うため、引数によらず常に共通ヒープの空きを返します。

4.3.23 FRND

書式：FRND(<値>) 高速な実数乱数を返します。乱数の状態は RND と共用されます。MSX-BACON 独自の拡張で、MSX-BASIC インタプリタには関数として存在しません。

4.3.24 HEX$

書式：HEX$(<値>) <値>を 16 進数表記の文字列に変換して返します。

4.3.25 HEXCHR$

書式：HEXCHR$(<16 進数文字列>) 16 進数文字列を 2 桁で 1 バイトとみなして文字列に変換して返します。CHRHEX$ の逆変換で、MSX-BACON 独自の関数です。

4.3.26 INKEY$

書式：INKEY$ キーバッファに入力済みの文字があれば最も古い 1 文字を、なければ空文字列を返します。括弧は付けません。

4.3.27 INP

書式：INP( ) 指定した I/O ポートを読み取り、その値を返します。

4.3.28 INPUT$

書式：INPUT$(<文字数>) / INPUT$(<文字数>, #<ファイル番号>) キーボードから<文字数>分のキー入力を受け付け、入力された文字列を返します。ファイル形では、指定ファイルから<文字数>分を読み取って返します。

4.3.29 INSTR

書式：INSTR(<文字列 1>, <文字列 2>) / INSTR(<開始位置>, <文字列 1>, <文字列 2>) <文字列 1>の中で<文字列 2>が最初に現れる位置を返します。先頭は 1 で、見つからなければ 0、<文字列 2>が空文字列なら 1 を返します。3 引数形では、<開始位置>から検索を始めます。

4.3.30 INT

書式：INT(<値>) <値>を超えない最大の整数を返します（小数部を切り捨てます）。戻り値の型は引数と同じです。

4.3.31 IRND

書式：IRND() -32768～32767 の範囲の整数乱数を返します。MSX-BACON 独自の関数で、種は IRANDOMIZE 文で設定します。

4.3.32 LEFT$

書式：LEFT$(<文字列>, <長さ>) <文字列>の左から<長さ>文字分を切り出して返します。<長さ>が文字列より大きい場合は文字列全体、0 の場合は空文字列を返します。制限：<長さ>に負の値や 256 以上の値を指定した場合の動作は不定です。

4.3.33 LEN

書式：LEN(<文字列>) / LEN(<数値変数>) 文字列の長さ（文字数）を返します。数値変数を渡した場合は、その型

のサイズ（整数型は 2、単精度実数型は 4、倍精度実数型は 8）を返します。数値変数形は MSX-BACON 独自の拡張で、インタプリタではエラーになります。

4.3.34 LOC

書式：LOC(<ファイル番号>) ファイルの現在位置を返します。シーケンシャルファイルでは、実際に消費したバイト数が返ります。ランダムアクセスファイルでは、現在位置のレコード番号が返ります。

4.3.35 LOF

書式：LOF(<ファイル番号>) ファイルの長さを返します。シーケンシャルファイルではバイト数、ランダムアクセスファイルではレコード数です。

4.3.36 LOG

書式：LOG(<値>) <値>の自然対数を返します。戻り値は倍精度実数です。

4.3.37 LPOS

書式：LPOS(<値>) プリンタの現在位置（行内の桁位置）を返します。引数はダミーで、値は使われません。

4.3.38 MAX

書式：MAX(<数値>, <数値>[, <数値> ...]) 引数のうち最大のものを返します。引数は 2 個以上指定できます。戻り値の型は、最も精度の高い引数の型になります。MSX-BACON 独自の拡張で、MSX-BASIC インタプリタには存在しません。

4.3.39 MID$

書式：MID$(<文字列>, <開始位置> [, <文字数>]) <文字列>の<開始位置>文字目（先頭は 1）から<文字数>文字分を切り出して返します。<文字数>を省略すると最後まで切り出します。制限：開始位置や文字数に負の値・256 以上の値を指定した場合の動作は不定です。

4.3.40 MKD$

書式：MKD$(<数値>) <数値>を倍精度実数としたときのメモリイメージそのもの（8 文字）を文字列として返します。CVD の逆変換です。

4.3.41 MKI$

書式：MKI$(<数値>) <数値>を整数としたときのメモリイメージそのもの（2 文字）を文字列として返します。CVI の逆変換です。

4.3.42 MKS$

書式：MKS$(<数値>) <数値>を単精度実数としたときのメモリイメージそのもの（4 文字）を文字列として返します。CVS の逆変換です。

4.3.43 MSXVER

書式：MSXVER() 機種の MSX バージョンを返します。MSX1 なら 0、MSX2 なら 1、MSX2+なら 2、MSXturboR なら 3 です。MSX-BACON 独自の関数です。

4.3.44 OCT$

書式：OCT$(<値>) <値>を 8 進数表記の文字列に変換して返します。

4.3.45 PAD

書式：PAD(<番号>) マウス・トラックボール・タッチパッドの情報を返します。制限：<番号>は 8～19 のみ有効です。

4.3.46 PDL

書式：PDL(<番号>) パドル（ジョイスティックのアナログ入力）の値を 0～255 で返します。

4.3.47 PEEK

書式：PEEK(<アドレス>) メモリの<アドレス>から 1 バイトを読み取り、0～255 の値を返します。

4.3.48 PEEKW

書式：PEEKW(<アドレス>) メモリの<アドレス>から 2 バイトを読み取り、値を返します。MSX-BACON 独自の関数です。

4.3.49 PEEKS$

書式：PEEKS$(<アドレス>, <サイズ>) メモリの<アドレス>から<サイズ>バイトを読み取り、文字列として返します。MSX-BACON 独自の関数です。制限：<サイズ>は 0～255 です。それ以外を指定した場合の動作は保証されません。

4.3.50 PLAY

書式：PLAY(<チャンネル番号>) 指定チャンネルの演奏状態を返します。演奏中かどうかを判定できます。

4.3.51 POINT

書式：POINT(,) 座標 (X,Y) の色を返します。制限：SCREEN 0、SCREEN 1 ではグラフィック座標ではなくテキスト座標を指定し、その位置の文字コードが返ります。

4.3.52 POS

書式：POS(<値>) 画面上のカーソルの現在位置（桁）を返します。引数はダミーで、値は使われません。

4.3.53 RIGHT$

書式：RIGHT$(<文字列>, <長さ>) <文字列>の右から<長さ>文字分を切り出して返します。<長さ>が文字列より大きい場合は文字列全体、0 の場合は空文字列を返します。制限：<長さ>に負の値や 256 以上の値を指定した場合の動作は不定です。

4.3.54 RND

書式：RND(<値>) 0 以上 1 未満の乱数を倍精度実数で返します。MSX-BASIC と互換の乱数列が得られます。制限：引数の省略や括弧なしの RND は使えません。

4.3.55 SGN

書式：SGN(<値>) <値>の符号を返します。負なら -1、0 なら 0、正なら 1 です。

4.3.56 SIN

書式：SIN(<値>) <値>（ラジアン）のサインを返します。戻り値は倍精度実数です。

4.3.57 SPACE$

書式：SPACE$(<値>) 半角スペースを<値>個つなげた文字列を返します。

4.3.58 SPC

書式：SPC(<式>) PRINT 文の中で使い、<式>の数だけ空白を出力してカーソルを進めます。式の中で使うと、空白が<式>の数だけ並んだ文字列になります（式中での利用は MSX-BACON 独自の拡張です）。TAB は引き続き PRINT 文専用です。

4.3.59 SPRITE$

書式：SPRITE$(<スプライト番号>) = <文字列> / <文字列変数> = SPRITE$(<スプライト番号>) スプライトパターンを文字列で設定します（代入形）。読み取り形では、現在のスプライトパターンを文字列として取得できます。文字列の長さはスプライトサイズに従い、8×8 では 8 文字、16×16 では 32 文字です。

4.3.60 SQR

書式：SQR(<値>) <値>の平方根を返します。戻り値は倍精度実数です。引数が定数の場合は、コンパイル時に計算されます。注記：配布された v0.9 では、コンパイラ内部に SQR の式ハンドラは存在したものの、式の振り分けに登録されていなかったため、SQR( ) を書くと Syntax error になっていました。v0.9.1 では通常どおり使えます。

4.3.61 STICK

書式：STICK(<番号>) 方向入力の状態を 0〜8 で返します。<番号>は 0 がカーソルキー、1 と 2 がジョイスティックです。0 は無入力、1〜8 が上から時計回りの方向です。

4.3.62 STR$

書式：STR$(<式>) 数値式の評価結果を文字列に変換して返します。正の数では先頭に空白が付きます。

4.3.63 STRIG

書式：STRIG(<番号>) トリガーボタンの状態を返します。<番号>は 0 がスペースキー、1〜4 がジョイスティックのボタンで、押されていれば -1、押されていなければ 0 です。

4.3.64 STRING$

書式：STRING$(<数>, <文字列式>) / STRING$(<数>, <文字コード>) 同じ文字を<数>個つなげた文字列を返します。文字は文字列式の先頭文字、または文字コードで指定します。

4.3.65 TAB

書式：TAB(<式>) PRINT 文の中で使い、カーソルを<式>で指定した桁位置へ進めます。制限：PRINT 文の中でしか使えません。式の中では使えず、PRINT # 文では何も出力されません。

4.3.66 TAN

書式：TAN(<値>) <値>（ラジアン）のタンジェントを返します。戻り値は倍精度実数です。

4.3.67 TIME

書式：TIME 1/60 秒ごとに増える 16 ビットのタイマー値を返します。括弧は付けません。TIME = <式> と代入してタイマー値を設定することもできます。

4.3.68 USR

書式：USR<番号> [<型識別子>] DEF USR で指定した機械語ルーチンを呼び出し、その戻り値を返します。<番号>は 0～9 で、省略すると 0 です。引数は VALTYP と DAC に MSX-BASIC と同じ形式で渡されます。制限：戻り値は既定では整数として扱われます。他の型で受け取る場合は呼び出しの直後に % ! # $ の型識別子を付けてください（MSX-BACON 独自の拡張）。

4.3.69 VAL

書式：VAL(<文字列>) 文字列を数値に変換して返します。&H などで始まる 16 進数などの表記も変換できます。数値として読めない場合は 0 を返します。

4.3.70 VARPTR

書式：VARPTR(<変数名>) / VARPTR(#<ファイル番号>) 変数の値が格納されているメモリ上のアドレスを返します。#<ファイル番号> の形では、#0 なら作業用 FCB、#1～#15 なら対応するファイル番号の FCB のアドレスを返します。

4.3.71 VDP

書式：VDP(<レジスタ番号>) VDP レジスタの値を読み取って返します。コントロールレジスタについては最後に書き込んだ値の控えが返ります。VDP(<レジスタ番号>) = <式> と代入して書き込むこともできます。制限：範囲外の番号を指定してもエラーにならず、不定な値が返ったり何も起こらなかったりします。代入では値の下位 8 ビットだけが有効です。

4.3.72 VPEEK

書式：VPEEK(<アドレス>) VRAM の<アドレス>から 1 バイトを読み取り、値を返します。

4.3.73 VPEEKS$

書式：VPEEKS$(<アドレス>, <サイズ>) VRAM の<アドレス>から<サイズ>バイトを読み取り、文字列として返します。MSX-BACON 独自の関数です。制限：<サイズ>は 0～255 です。それ以外を指定した場合の動作は保証されません。

4.4 演算子

式に使える演算子と、その優先順位（上が高い）は次のとおりである。これはコンパイラの式パーサの構造そのもので、MSX-BASIC と一致する。

優先順位	演算子	内容
1（最高）	^	べき乗。右辺が整数で 0～255 の範囲なら整数べき乗として計算する
2	単項 + / -	符号
3	* /	乗算・除算（実数除算）
4	\	整数除算。両辺を整数化してから割る
5	MOD	剰余。両辺を整数化してから計算する
6	+ -	加算・減算。+ は両辺が文字列なら連結

優先順位	演算子	内容
7	= <> >< < > <= >= (=< => も可)	比較。結果は整数型で真が -1、偽が 0
8	NOT	ビット単位の否定(単項)
9	AND	ビット単位の論理積
10	OR	ビット単位の論理和
11	XOR	ビット単位の排他的論理和
12	IMP	含意((NOT 左辺) OR 右辺)
13(最低)	EQV	等価(NOT (左辺 XOR 右辺))

- ***数値同士の演算では、両辺の型が倍精度実数 > 単精度実数 > 整数の順に揃えられてから計算される。整数同士の + - * は整数演算になる。***
- ***比較は数値同士のほか、文字列同士(辞書順)でも行える。***
- ***論理演算子は条件の短絡評価を行わない。両辺は常に評価される。***
- ***高速な実数乱数が必要な場合は FRND も使える(4.3 節。MSX-BACON 独自の拡張)。***

命令索引

本書の重要な用語と、その用語を主に説明しているページの一覧である。ページ番号はリンクになっており、クリックすると該当箇所へ移動する。

用語	ページ
ABS	50
ASC	51
ATN	51
AUTOEXEC.BAS	11
BACONLDR.BIN	10
BACONLIB	10
BASE	51
BEEP	36
BIN$	51
BINDATA	36
BLOAD	36
BSAVE	36
CALL IOTGET	36
CALL IOTINIT	37
CALL IOTPUT	37

用語	ページ
CALL TURBO	37
CDBL	51
CHR$	51
CHRHEX$	51
CINT	51
CIRCLE	37
CLEAR	37
CLOSE	37
CLS	37
COLOR	38
COLOR SPRITE	38
COPY	38
COS	52
CSNG	52
CSRLIN	52
CVD	52
CVI	52
CVS	52

用語	ページ
DATA	38
DEF FN	38
DEF USR	38
DEFDBL	39
DEFINT	39
DEFSNG	39
DEFSTR	39
DIM	39
DRAW	39
DSKF	52
END	39
EOF	52
ERASE	40
ERL	53
ERR	53
ERROR	40
EXP	53
FIELD	40

用語	ページ
FILES	40
FIX	53
FOR	40
FRE	53
FRND	53
GET #	40
GET DATE	41
GET TIME	41
GOSUB	41
GOTO	41
HEX$	53
HEXCHR$	53
HEXDATA	41
IF	41
INKEY$	54
INP	54
INPUT	41
INPUT #	41

用語	ページ
INPUT$	54
INSTR	54
INT	54
IRANDOMIZE	42
IRND	54
KEY	42
KEY(n)	42
KILL	42
LEFT$	54
LEN	54
LET	42
LINE	42
LINE INPUT	42
LOAD	43
LOC	55
LOCATE	43
LOF	55
LOG	55

用語	ページ
LPOS	55
LPRINT	43
LSET	43
MAX	55
MAXFILES	43
MID$	55
MID$（文）	43
MKD$	55
MKI$	56
MKS$	56
MSX-BASIC	3
MSXVER	56
NAME	44
NEXT	40
OCT$	56
ON ERROR GOTO	44
ON GOSUB	44
ON GOTO	44

用語	ページ
ON INTERVAL GOSUB	44
ON KEY GOSUB	44
ON SPRITE GOSUB	44
ON STOP GOSUB	45
ON STRIG GOSUB	45
OPEN	45
openMSX	11
OUT	45
PAD	56
PAINT	45
PDL	56
PEEK	56
PEEKS$	57
PEEKW	56
PLAY	45
PLAY(n)	57
POINT	57
POKE	45

用語	ページ
POKES	45
POKEW	46
POS	57
PRESET	46
PRINT	46
PRINT #	46
PRINT USING	46
PSET	46
PUT #	46
PUT SPRITE	46
READ	47
REM	47
RESTORE	47
RESUME	47
RETURN	47
RIGHT$	57
RND	57
RSET	47

用語	ページ
RUN	47
SCREEN	48
SET ADJUST	48
SET ADJUSTS	48
SET BEEP	48
SET DATE	48
SET PAGE	48
SET PROMPT	48
SET SCREEN	48
SET SCROLL	49
SET TIME	48
SET TITLE	49
SGN	57
SIN	57
SOUND	49
SPACE$	58
SPC	58
SPRITE ON/OFF/STOP	49

用語	ページ
SPRITE$	58
SPRITE$（代入）	49
SQR	58
STICK	58
STOP ON/OFF/STOP	49
STR$	58
STRIG	58
STRIG(n)	49
STRING$	59
SWAP	49
TAB	59
TAN	59
TIME	59
TIME（代入）	50
USR	59
VAL	59
VARPTR	59
VDP	60

用語	ページ
VDP(代入)	50
VPEEK	60
VPEEKS$	60
VPOKE	50
VPOKES	50
WAIT	50
WIDTH	50
ZMA	3
開始アドレス	6
型	13
コンパイルエラー	7
コンパイルモード	6
最適化	6
実行時エラー	12
スタック	6
対応状況一覧	20
バージョン履歴	8
ラベル	13

用語	ページ
割り込み	18

MSX-BACON USER'S MANUAL

2026年10月31日発行

著作　原 貴幸 (HRA!)